

创意与技术说明

项目：五子棋（Gomoku）• Godot 4 图形化对战程序
版本：v1.3.0 | 引擎：Godot 4.7.2-stable | 语言：GDScript

一、设计理念

传统五子棋程序往往停在「能下、能判断胜负」的功能层面。本项目的目标是在**功能之外**再往前走三步：

1. **一盘多用** —— 不做死一个 15×15 五连，而是把棋盘尺寸、连珠数、难度、先后手全部参数化，同一套程序覆盖多种玩法；
2. **视觉有性格** —— 不满足于灰底黑线，用古风木色羊皮纸质感建立辨识度；
3. **算法会适配** —— AI 不是三档固定深度的静态程序，而是**随规则自我调整搜索策略与评估权重**。

二、创意点

创意 1 • 古风木色视觉

- **羊皮纸回纹背景**：整幅木色羊皮纸底图铺满窗口，保留传统回纹外框
- **半透明暖木棋盘**：棋盘不是不透明色块，而是以 $\alpha = 0.55$ 的径向渐变暖木叠加在羊皮纸上，让底层纹理透出来；外框做双层（外粗深棕 + 内细浅棕）模拟木牌厚度
- **毛笔字标题**：字体族优先取 **楷体**（KaiTi → 楷体 → STKaiti → SimSun → 宋体 → ...），标题加粗放大到 64px，配合 **#3E2723** 深棕文字
- **半透明圆角面板**：所有设置分组用 **#FFFFFFAA** 底 + 8px 圆角 + 棕金描边 + 投影，从木纹背景中清晰浮起

创意 2 • 零素材依赖（背景图除外）

除背景图外，所有视觉与听觉元素均由代码生成，仓库里没有任何 ttf / wav / shader 文件：

元素	实现方式
木纹渐变	GradientTexture2D 径向填充 + 固定种子生成的细木纹线

中文字体	<code>SystemFont</code> + 8 级回退链（内置字体不含 CJK，直接用会显示方框）
落子音效	<code>AudioStreamWAV</code> 现场合成：频率 500→72 Hz 下滑 + 指数衰减包络，0.12 s / 22050 Hz
棋子立体感	外阴影 + 边缘压暗弧 + 双层高光，全部由 <code>draw_circle</code> / <code>draw_arc</code> 绘制
圆角面板 / 按钮	<code>StyleBoxFlat</code> 程序构造

这么做的好处：仓库体积小、跨平台不依赖系统字体安装情况、任何参数都能在代码里调。

创意 3 • 规则自适应 AI（本项目最大的算法创意）

多数五子棋 AI 把「15 路 5 连」写死在代码里。本项目让 AI 读取实际规则并调整自身：

- 棋盘尺寸 → 搜索深度：9 路搜 5 层、19 路只搜 2 层 —— 大棋盘分支爆炸必须收敛
- 连珠数 → 评估权重：所有棋型判定相对于 `win_count`，且活三权重动态变化 —— 4 连规则下活三从 1000 分提升到 10000 分（与冲四同级），因为 4 连时「活三 → 活四 → 取胜」只需两步

这两个机制让同一份算法代码在 9×9/3 连和 19×19/6 连下都保持合理强度与耗时。

创意 4 • 参数化玩法面板

右侧面板把四个玩法维度全部暴露给玩家，且改动即时生效：

控件	范围
棋盘大小	9 / 11 / 13 / 15 / 17 / 19（滑动条步长固定为 2，只产生奇数）
连珠数	5 / 6
AI 难度	简单 / 普通 / 困难
执子	执黑 • 先行 / 执白 • 后行（选执白后 AI 自动先手）

创意 5 • 细节交互

交互	实现
悬停幽灵棋子	鼠标停在空交叉点时绘制 0.4 透明度的当前回合色棋子（带描边保证木色底上可见）
Q 弹落子	棋子 $0.5\times \rightarrow 1.2\times \rightarrow 1.0\times$ ， <code>TRANS_QUAD</code> 起跳 +

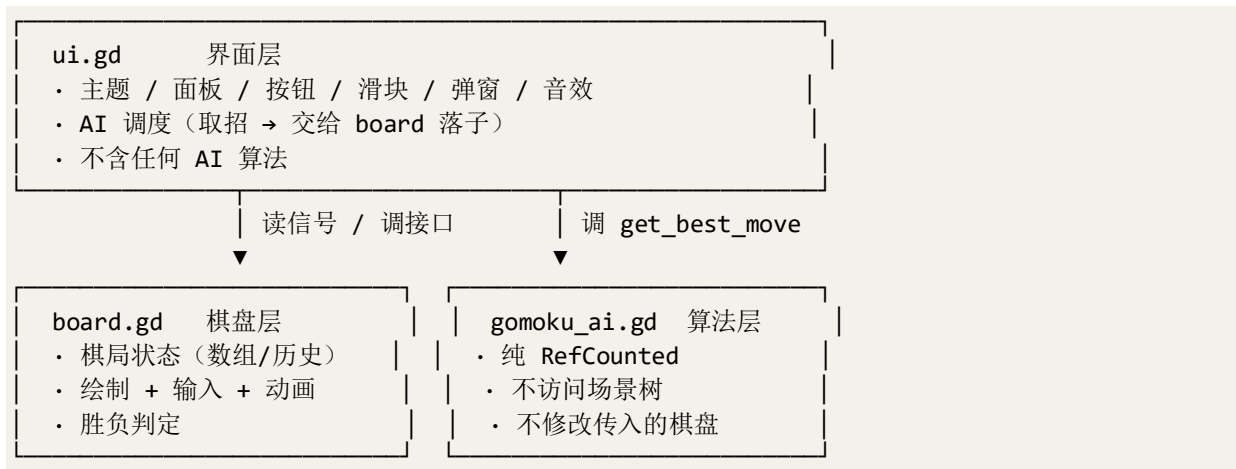
	TRANS_BACK 回弹, 260 ms
最后一手标记	最新落子位置画红点, 并随该棋子的弹跳一起缩放
胜利连线闪烁	回溯出真正连成 win_count 子的那条线, 做呼吸式 alpha 闪烁
胜负弹窗	淡入 + TRANS_BACK 缩放弹出 (0.72 → 1.0)
滑动条防抖	拖动中只更新文字, drag_ended 后才重建棋局
开始界面	标题卡 + 「开始游戏」按钮, 淡出放大后进入对局; 未开始前棋盘锁定、AI 不动

创意 6 • 开始界面与「先配置后开局」

开始界面不完全遮挡右侧设置面板 —— 玩家可以先调好棋盘大小、连珠数、难度和执子, 再点「开始游戏」。未开始期间棋盘输入被锁死, AI 也不会抢跑。

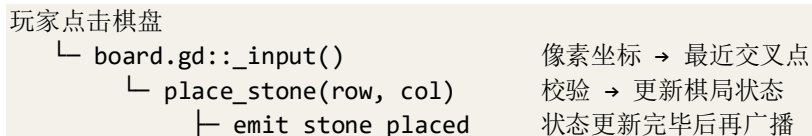
三、技术架构

3.1 三层解耦



关键约束：三层之间只通过信号与公开方法通信, gomoku_ai.gd 可以被任意替换而不影响其它两层。

3.2 数据流



```

└─ emit game_over      仅终局时

ui.gd::_on_stone_placed()
├─ 刷新轮次 / 步数 / 按钮
└─ 轮到 AI → get_best_move() → board.set_ai_move()

board.gd 自身监听 stone_placed 驱动落子弹跳（视觉层）

```

3.3 视觉状态与棋局状态严格分离

board.gd 中新增独立的「视觉状态」区块，只读棋局数据、从不写回：

```

# 视觉状态（仅服务渲染与动画，不参与任何棋局判定）
var _stone_scale: Dictionary = {}      # 落子弹跳缩放
var _hover_cell: Vector2i = ...        # 悬停格
var _win_line: Array = []              # 获胜连线
var _win_phase: float = 0.0           # 闪烁相位

```

动画通过监听已有的 stone_placed 信号驱动，完全不侵入落子逻辑。

3.4 统一规则入口

棋盘尺寸 / 连珠数 / 难度只能通过一个方法变更：

```

func set_rules(new_size: int, new_win: int, new_level: int = -1) -> void

```

它负责：夹取参数 → 偶数取奇 → 重算几何 → 清空视觉状态 → 重开一局。禁止绕过它直接改`board\_size`，避免尺寸变量与棋盘数组不同步。

四、技术难点与解决

开发过程中遇到并修复的 8 个真实问题，按严重程度排列：

#	问题	根因	解决方案
1	棋盘完全点不动	根节点 <code>Control</code> 默认 <code>mouse_filter = STOP</code> ，盖住整个视口， <code>gui_get_hovered_control()</code> 永远非空，落子的 UI 守卫把所有点击都拒了	根节点与所有非交互控件统一设为 <code>IGNORE</code> 。用 对照实验确认 ：改回默认后点击全部被拒
2	调整棋盘大小后棋子不显示	19 路时悬停格为 (17, 17)，切到 9 路后同一帧的 <code>_draw()</code> 仍用旧坐标	三重防护： <code>set_rules()</code> 清空悬停格 + 绘制函数加双重边界判断 + 按数

		索引新数组 → Out of bounds → 报错中断整个 <code>`_draw()`</code> ，其后的画棋子函数被跳过	组实际尺寸遍历
3	AI 永远返回 (-1, -1)	内部棋盘 <code>_grid.fill(WALL)</code> 把整个数组填成了哨兵值，导致所有空点都被判为「非空」，候选列表恒为空	改为只把外圈 PAD 层设为 WALL ，内部填 EMPTY
4	AI 从不被触发	<code>stone_placed</code> 在 <code>current_player</code> 更新之前发出，UI 读到的是落子方的旧值，永远判定「还没轮到 AI」	先把状态改完再广播信号
5	AI 生成后无法解析	<code>PackedInt32Array</code> 没有 <code>pop_back()</code> （只有 <code>Array</code> 有）	撤销栈改为「读栈顶索引 + 最后一次 <code>resize()</code> 截断」
6	调参被静默丢弃	<code>_apply_slider_rules()</code> 在 AI 思考时直接 <code>return</code> ，滑块显示 19 但棋盘还是 15，且永不同步	改为重启防抖定时器，稍后重试
7	滑动条拖了不生效	只监听 <code>drag_ended</code> 会漏掉点击滑轨、方向键这些不发该信号的改值方式	<code>value_changed</code> 起一次性定时器做防抖 + <code>drag_ended</code> 立即应用
8	动画被压缩掉大半	场景加载首帧 <code>delta</code> 高达 60 ms（渐变纹理构建等），260 ms 的弹跳动画 70 ms 就跑完了	定位为「首帧大 <code>delta</code> 」而非参数错误；验证动画时先预热 1 秒再采样

特别说明：问题 2 的定位过程

这个问题最初表现为「调大小后显示坏了」，很容易误判为渲染残留状态。我们用像素级 A/B 对比先排除了这个可能：

构造「一开始就是 9 路」与「15 → 19 → 9 切换过去」两个场景，逐像素比较 —— 76 万像素零差异，证明切换不残留任何渲染状态。

排除了错误假设后，才通过带渲染运行捕获到真实报错栈，定位到悬停坐标越界。如果按最初的猜测去改，很可能改错地方。

五、工程实践

5.1 三层验证体系

层级	手段	覆盖内容
静态	<code>godot --headless --check-only --script</code>	三个脚本语法
运行期	<code>--headless --quit-after N</code> 跑主场景抓 ERROR	场景解析、节点路径、空引用
行为	无头 / 带渲染跑测试场景 + 断言	棋局逻辑、鼠标输入、AI 战术、参数化、UI 流程
视觉	带渲染截图 + 逐帧采样	木纹、棋子、悬停、动画轨迹、弹窗

5.2 累计测试规模

测试	断言数	结果
棋盘逻辑（轮次 / 悔棋 / 连子 / 非法落子）	33	√
鼠标输入（真实 <code>push_input</code> 管线、坐标吸附、按钮防误触）	10	√
AI 战术（空盘天元 / 冲四补五 / 堵冲四 / 堵活四 / 三档难度）	20	√
玩法参数化（7 组尺寸 × 连珠组合）	77	√
v1.2.1 修正（连珠数下限 / 尺寸切换 / 执子先后手）	21	√
v1.3.0（开始界面 / 奇数尺寸）	24	√
自对弈压力测试	2 局	√

5.3 关键验证技巧

- 带渲染才能测鼠标：无头模式下视口尺寸与带渲染不同，stretch 变换会让 `push_input` 的坐标偏移，点击测试会假失败
- `Input.warp_mouse()` 不触发 Control 悬停：它只改光标位置，不产生 GUI 的 motion 事件。验证 hover 必须 `get_viewport().push_input(InputEventMouseMotion)`

- 动画验证要采样轨迹：只截图看不出 260 ms 的曲线，逐帧记录缩放值才能确认 0.5 → 1.2 → 1.0 完整走完（实测峰值 1.175、回弹过冲至 0.980 再归位）

六、可扩展性

想做的事	改动位置
换一套 AI	替换 gomoku_ai.gd，保持 4 个公开方法签名即可
加双人对战	ui.gd 里不触发 AI 回合即可（AI 调度与棋盘层已解耦）
加音效 / 背景音乐	ui.gd::_build_sound() 已封装音频播放器
改视觉风格	ui.gd::_build_theme() 集中管理颜色与样式；棋盘绘制在 board.gd::_draw_*
加禁手规则	在 board.gd::_has_win() 与 AI 的 _makes_five() 中扩展
加网络对战	board.gd 已把「落子」与「渲染」分离，可只同步落子事件

七、项目结构

game3/	
├─ project.godot	# 主场景 / 窗口 1000x760 / 版本号
├─ README.md	# 项目说明
├─ CLAUDE.md	# 引擎与代码约定参考（含 18 条已修复的坑）
├─ scenes/	
│ └─ Main.tscn	# 主场景：BoardUI + 背景 + 开始界面 + 右侧面板
├─ scripts/	
│ └─ board.gd	# 棋盘层：参数化几何 + 绘制 + 状态 + 输入
│ └─ ui.gd	# 界面层：主题 / 面板 / 弹窗 / 音效 / AI 调度
│ └─ gomoku_ai.gd	# 算法层：纯 RefCounted，规则自适应
├─ assets/ui/wood_bg.png	# 木色羊皮纸背景（唯一外部素材）
└─ docs/	
│ └─ screenshot.png	# 对局主图
│ └─ start_screen.png	# 开始界面
│ └─ ui_9x9.png / ui_19x19.png	# 不同棋盘规格
│ └─ ui_win.png	# 胜利界面
│ └─ UI_OPTIMIZATION.md	# v1.1 UI 优化说明
│ └─ V12_UPDATE.md	# v1.2 / v1.2.1 参数化与修正说明
│ └─ competition/	# 比赛提交材料
│ └─ README.md	
│ └─ 01-gameplay-and-algorithm.md	
│ └─ 02-innovation-and-technology.md	

八、运行方式

```
# 用 Godot 4.5+ 打开本目录，或命令行直接运行  
godot --path . --rendering-driver opengl3
```

主场景已配置为 `res://scenes/Main.tscn`，启动即进入开始界面。